DESIGN FOR CS 498 HOMEWORK 3

PART 1: WIDE-COLUMN SCHEMA DESIGN AND SHARDING

1. PROPOSED ROW KEY

I would use the following composite row key:

‘bucket#city#make#model_year#vin10‘

Where:

* ‘bucket‘ is a short hash prefix derived from ‘city + make + vin10‘ and spread across a fixed number of buckets such as 16.
* ‘city‘ keeps records for the same city logically grouped.
* ‘make‘ keeps vehicle brand information near the city partition.
* ‘model_year‘ helps organize vehicles for common analytical filters.
* ‘vin10‘ preserves uniqueness at the row level.

This design avoids hotspotting because high-volume inserts do not all land in one contiguous key range. The hash bucket spreads incoming writes across multiple tablets even when the workload is skewed toward popular values such as ‘Seattle‘ or ‘TESLA‘. That improves load balancing and reduces the chance that a single node becomes the write bottleneck.

The trade-off is query complexity. Since the bucket appears first, a query for one city or one make must fan out across all buckets rather than scanning a single prefix. However, this is still much more efficient than a purely random key because ‘city‘, ‘make‘, and ‘model_year‘ remain clustered inside each bucket. In practice, the application can issue parallel scans over the small fixed bucket set. If city-only or make-only queries become dominant, I would add a secondary index table keyed specifically for those access patterns.

2. COLUMN FAMILY DESIGN

I would group the EV dataset into the following column families:

1. ‘identity‘
   - ‘vin10‘
   - ‘dol_vehicle_id‘

2. ‘vehicle_specs‘
   - ‘model_year‘
   - ‘make‘
   - ‘model‘
   - ‘electric_vehicle_type‘
   - ‘electric_range‘
   - ‘base_msrp‘

3. ‘location‘
   - ‘county‘
   - ‘city‘
   - ‘state‘
   - ‘postal_code‘
   - ‘vehicle_location‘
   - ‘legislative_district‘
   - ‘census_tract‘

4. ‘programs‘
   - ‘cafv_eligibility‘
   - ‘electric_utility‘

This grouping improves distributed read performance because Bigtable stores data by column family. Queries that only need vehicle brand, model year, and range can read the ‘vehicle_specs‘ family without touching location or utility-related data. Likewise, location-heavy analytical queries can focus on the ‘location‘ family. That reduces unnecessary disk I/O, network transfer, and memory pressure on each distributed node.

3. HASHED SHARD KEY VS. RANGE-BASED SHARD KEY

HASHED SHARD KEY ON VIN

Pros:

* Excellent write distribution across the cluster.
* Minimizes hotspotting because adjacent VIN values do not land on the same shard.
* Good overall cluster balance for high-ingest workloads.

Cons:

* Poor query routing for range queries and filtering by business attributes.
* Queries by ‘Model Year‘, ‘City‘, or ‘Make‘ often need to scatter to all shards.
* Harder to exploit locality for analytical scans.

RANGE-BASED SHARD KEY ON MODEL YEAR

Pros:

* Very efficient routing for year-based queries such as ‘Model Year >= 2022‘.
* Keeps nearby years on nearby shards, which helps temporal analytics.
* Easier to understand operationally when the workload is naturally time-based.

Cons:

* Popular or recent years can become hotspots.
* Skewed inserts may overload a subset of shards.

* Rebalancing becomes more likely when new years attract a disproportionate share of writes.

Overall, a hashed key is better for even cluster performance and write scalability, while a range-based key is better for query locality when the application frequently filters on the shard key itself.

PART 4: PERFORMANCE ANALYSIS

BENCHMARK METHOD

I used ‘benchmark.py‘ to send 50 POST requests to ‘/insert-fast‘ and 50 POST requests to ‘/insert-safe‘, then computed the average latency for each endpoint.

Example command:

‘‘‘bash
python3 benchmark.py --base-url http://YOUR_VM_IP:5000
‘‘‘

AVERAGE LATENCY RESULTS

* Average latency for ‘/insert-fast‘: ‘_____ ms‘
* Average latency for ‘/insert-safe‘: ‘_____ ms‘

The expected result is that ‘/insert-fast‘ is faster because ‘WriteConcern(w=1)‘ only waits for acknowledgment from the primary node, while ‘/insert-safe‘ uses ‘WriteConcern(w="majority")‘ and must wait for replication to the majority of the replica set before returning success.

CAP-THEOREM-BASED ANALYSIS

A good real-world example for choosing the faster primary-only write is user-behavior analytics, such as recording ad impressions, page-view events, or recommendation-click logs. These events are high volume and latency sensitive, and losing a very small number of records during a rare primary crash is usually acceptable. In that case, lower write latency and higher throughput are more valuable than strict durability for every single event.